HTTP-клиент и сервер с графическим интерфейсом на Python 2
1. Постановка задачи 2
2. Требования 2
2.1. Технологический стек 2
2.2. HTTP-клиент 2
2.3. HTTP-сервер 3
2.4. Графический интерфейс 4
2.5. Общие требования 5
3. Предлагаемая архитектура 6
4. Этапы выполнения 8
5. Тесты 9
5.1. Unit-тесты 9
5.2. Интеграционные тесты 10
5.3. GUI-тесты (ручные, по скриншотам) 10
5.4. Команды 10
6. Что должно быть на скриншотах 11
7. Вопросы на защиту 12
7.1. Про HTTP 12
7.2. Про сокеты 12
7.3. Про Python и ООП 12
7.4. Про GUI и многопоточность 13
7.5. Про безопасность 13
8. Критерии оценки 13

HTTP-клиент и сервер с графическим интерфейсом на Python

1. Постановка задачи

Разработать десктопное приложение на Python с графическим интерфейсом, состоящее из двух модулей:

1. **HTTP-клиент** — вкладка в GUI, где пользователь вводит URL, выбирает метод (GET/POST), заголовки и тело запроса, нажимает «Отправить» и видит ответ (статус, заголовки, тело).
2. **HTTP-сервер** — вкладка в GUI, где пользователь задаёт порт, нажимает «Запустить», и видит лог входящих запросов. Сервер обрабатывает GET и POST.

Клиент и сервер работают в одном приложении (два окна или две вкладки). Сервер должен работать в отдельном потоке, чтобы GUI не замерзал.

Цель: понять HTTP «под капотом», спроектировать ООП-архитектуру и освоить многопоточность в GUI-приложении.

КОНФИДЕНЦИАЛЬНО

2. Требования

2.1. Технологический стек

Компонент	Разрешено	Запрещено
Язык	Python 3.10+	—
GUI	tkinter (стандарт) или PyQt5/6	—
Сеть	socket, ssl (для HTTPS — бонус)	requests, urllib3, aiohttp, httpx
Сервер	socket, threading	flask, fastapi, http.server
Парсинг	re, json, стандартная библиотека	beautifulsoup (не нужен для HTTP)
Тесты	unittest / pytest	—

2.2. HTTP-клиент

Функции:

Функция	Описание
get(url, headers=None)	GET-запрос
post(url, body, content_type, headers=None)	POST-запрос
set_timeout(ms)	таймаут соединения и чтения

follow_redirects(max)	следовать за 3xx (бонус)

Что надо уметь:

- Парсить URL: http://host:port/path?query → host, port, path, query.
- Поддерживать https:// через ssl (бонус +5).
- Читать ответ: статус-код, заголовки, тело.
- Обрабатывать Content-Length и Transfer-Encoding: chunked (chunked — бонус).
- Корректно закрывать сокет (контекстный менеджер with).
- Бросать исключения: ConnectionError, TimeoutError, ParseError, HttpError(status).
- **Не блокировать GUI:** запрос выполняется в отдельном потоке, результат возвращается через queue.Queue или root.after.

2.3. HTTP-сервер

КОНФИДЕНЦИАЛЬНО

Маршруты:

Маршрут	Метод	Ответ
/	GET	HTML-страница «Hello»
/echo	POST	эхо тела, Content-Type сохраняется
/time	GET	текущее время в JSON
/headers	GET	все заголовки запроса в JSON
/status/{code}	GET	ответ с указанным статус-кодом
*	любой	404 Not Found

Что надо уметь:

- Слушать порт (по умолчанию 8080), запускаться/останавливаться по кнопке.
- Принимать соединения в цикле в отдельном потоке.
- Парсить HTTP-запрос: метод, путь, версия, заголовки, тело.
- Формировать HTTP-ответ: статус-строка, заголовки (Content-Type, Content-Length, Date), тело.
- Обрабатывать Connection: close и keep-alive.
- Не падать на некорректных запросах → 400 Bad Request.

- Логировать запросы в GUI: метод, путь, статус, время обработки.
- Многопоточная обработка соединений (поток на соединение или пул) — **+10 баллов**.

2.4. Графический интерфейс

Минимальные требования:

Вкладка «Клиент»:

- Поле URL.
- Выпадающий список метода (GET/POST).
- Таблица заголовков (добавить/удалить строку: имя → значение).
- Многострочное поле тела (активно только для POST).
- Кнопка «Отправить».
- Кнопка «Очистить».
- Область ответа: статус, заголовки, тело (с подсветкой JSON/HTML).
- Индикатор загрузки (спиннер / «Отправка...»).

КОНФИДЕНЦИАЛЬНО

Вкладка «Сервер»:

- Поле порта.
- Кнопки «Запустить» / «Остановить».
- Индикатор состояния (зелёный/красный).
- Лог запросов (таблица: время, метод, путь, статус, мс).
- Кнопка «Очистить лог».
- Кнопка «Открыть в браузере» (открывает http://localhost:8080/).
- **Вкладка «Настройки» (опционально):**
- Таймаут по умолчанию.
- Тема (светлая/тёмная).
- Сохранение истории запросов в JSON.

Требования к GUI:

- Не замерзать во время запросов/работы сервера — всё сетевое в фоновых потоках.
- Обновление GUI только из главного потока (root.after для tkinter, сигналы/слоты для PyQt).
- Валидация ввода: URL, порт (1–65535), заголовки.
- Обработка ошибок: показать пользователю ConnectionError, TimeoutError и т.д. в области ответа, не в консоли.

2.5. Общие требования

- **ООП-архитектура:** минимум 6 классов, минимум 2 интерфейса (ABC).
- **RAII / контекстные менеджеры:** сокеты, файлы — через with.
- **Исключения:** для сетевых ошибок; никаких «магических» кодов возврата.
- **Без глобального состояния:** никаких global для сокетов.
- **Тесты:** unit + интеграционные (клиент ↔ сервер).
- **Логирование:** модуль logging, вывод в файл и в GUI.
- **Линтеры:** flake8, pylint — без ошибок (warning'и допустимы с обоснованием).

КОНФИДЕНЦИАЛЬНО

3. Предлагаемая архитектура

```
http_gui/
├── src/
│   ├── http/
│   │   ├── url.py            # Url, ParseError
│   │   ├── request.py        # Request
│   │   ├── response.py       # Response
│   │   ├── parser.py         # Parser (parse_request, parse_response)
│   │   └── serializer.py     # Serializer
│   ├── net/
│   │   ├── socket_wrapper.py   # Socket (context manager)
│   │   ├── tcp_client.py     # TcpClient
│   │   └── tcp_server.py     # TcpServer
│   ├── client/
│   │   └── http_client.py    # HttpClient
│   ├── server/
│   │   ├── http_server.py    # HttpServer (thread)
│   │   ├── router.py         # Router
│   │   └── handlers.py       # handlers: /, /echo, /time, /headers, /status
│   ├── gui/
│   │   ├── app.py            # main window, tabs
│   │   ├── client_tab.py     # ClientTab
│   │   ├── server_tab.py     # ServerTab
│   │   └── widgets.py        # HeaderTable, LogTable, ...
│   └── main.py               # entry point
├── tests/
│   ├── test_url.py
│   ├── test_parser.py
│   ├── test_serializer.py
│   └── test_integration.py
├── requirements.txt          # только pytest, flake8 (опционально)
├── Makefile
└── README.md
```

КОНФИДЕНЦИАЛЬНО

Ключевые интерфейсы (пример):

```
# src/http/request.py
from dataclasses import dataclass, field
from typing import Dict, Optional

@dataclass
class Request:
    method: str
    path: str
    version: str = "HTTP/1.1"
    headers: Dict[str, str] = field(default_factory=dict)
    body: bytes = b""

# src/http/response.py
@dataclass
class Response:
    status: int
    reason: str = ""
    version: str = "HTTP/1.1"
    headers: Dict[str, str] = field(default_factory=dict)
    body: bytes = b""

# src/net/socket_wrapper.py
import socket

class Socket:
    def __init__(self, sock: socket.socket):
        self._sock = sock

    def __enter__(self):
        return self

    def __exit__(self, *exc):
        self.close()

    def close(self):
        try:
            self._sock.close()
        finally:
            self._sock = None

    def send_all(self, data: bytes):
        self._sock.sendall(data)

    def recv(self, n: int) -> bytes:
        return self._sock.recv(n)

# src/server/router.py
from typing import Callable, Dict, Tuple
from ..http.request import Request
```

КОНФИДЕНЦИАЛЬНО

```
from ..http.response import Response

Handler = Callable[[Request], Response]

class Router:
    def __init__(self):
        self._routes: Dict[Tuple[str, str], Handler] = {}

    def route(self, method: str, path: str, handler: Handler):
        self._routes[(method.upper(), path)] = handler

    def dispatch(self, req: Request) -> Response:
        key = (req.method.upper(), req.path)
        if key in self._routes:
            return self._routes[key](req)
        # /status/{code}
        if req.path.startswith("/status/"):
            try:
                code = int(req.path.split("/")[-1])
                return Response(status=code, body=b"")
            except ValueError:
                pass
        return Response(status=404, reason="Not Found",
body=b"404")
```

КОНФИДЕНЦИАЛЬНО

4. Этапы выполнения

Этап 1 Парсинг и сериализация



Url: разобрать http://host:port/path?query.



Parser: разобрать сырой HTTP-запрос/ответ.



Serializer: собрать Request/Response в байты.



Тесты: 20+ unit-тестов, включая граничные случаи.

Этап 2. Сокеты и клиент



Socket — контекстный менеджер.

 TcpClient.connect — с таймаутом (socket.settimeout).

 HttpClient.get / post — собрать, отправить, прочитать, распарсить.

 **Тест:** GET http://example.com → 200.

Этап 3. Сервер

 TcpServer — bind, listen, accept.

 HttpServer — цикл в отдельном потоке (threading.Thread).

 Router + handlers.

 **Тест:** curl http://localhost:8080/ → 200.

остановка сервера — через флаг + закрытие сокета, не kill.

Этап 4 . GUI

КОНФИДЕНЦИАЛЬНО

 tkinter или PyQt.

 Вкладка «Клиент»: все поля, фоновый поток, root.after для обновления.

 Вкладка «Сервер»: запуск/остановка, лог через queue.Queue + root.after.

 Обработка ошибок в GUI.

Этап 5. Интеграция и качество

 Интеграционные тесты: сервер в потоке + клиент в тесте.

 TSan-аналог: python -X dev, threading без гонок (проверить логи).

 flake8, pylint — чисто.

 README с примерами.

 Отчёт с диаграммой классов.

5. Тесты

5.1. Unit-тесты

№	Тест	Что проверяет

U1	Url.parse("http://host:8080/path?q=1")	host, port, path, query
U2	Url.parse("http://host/path")	port=80
U3	Url.parse("garbage")	ParseError
U4	Parser.parse_request(b"GET / HTTP/1.1\r\n\r\n")	method=GET, path=/
U5	Parser.parse_request с телом и Content-Length	body прочитан
U6	Parser.parse_request без Host	ParseError / 400
U7	Serializer.serialize(Request)	заканчивается \r\n\r\n
U8	Serializer.serialize(Response)	есть Content-Length
U9	Response с chunked	тело собрано
U10	Заголовок > 8 КБ	ParseError (защита от overflow, ЛБ

5.2. Интеграционные тесты

№	Тест	Что проверяет
I1	Сервер в потоке, HttpClient.get("/")	200, тело «Hello»
I2	post("/echo", b"abc", "text/plain")	ответ «abc», Content-Type
I3	get("/status/404")	404
I4	get("/nonexistent")	404
I5	100 последовательных GET	все 200
I6	10 параллельных GET (потоки)	все 200
I7	Таймаут 100 мс, сервер «спит» 1 сек	TimeoutError
I8	Мусор вместо HTTP	400, сервер жив
I9	keep-alive — 3 запроса в одном соединении	все 200 (бонус)
I10	1000 запросов	нет падений, нет утечек

КОНФИДЕНЦИАЛЬНО

5.3. GUI-тесты (ручные, по скриншотам)

№	Тест	Что проверяет
G1	Ввод URL, GET, «Отправить»	ответ в области, GUI не замерзает
G2	POST с телом	эхо в ответе
G3	Неверный URL	ошибка в GUI, не в консоли
G4	Запуск сервера, curl из терминала	лог в GUI обновляется
G5	Остановка сервера	порт освобождён, curl не подключается
G6	10 параллельных запросов к серверу	лог без гонок

5.4. Команды

```
bash
# Установка (только тесты и линтеры)
pip install -r requirements.txt

# Запуск приложения
python -m src.main

# Unit-тесты
pytest tests/ -v

# Только интеграционные
pytest tests/test_integration.py -v

# Линтеры
flake8 src/ tests/
pylint src/

# Ручная проверка
python -m src.main &
curl -v http://localhost:8080/
curl -v -X POST -d "hello" http://localhost:8080/echo
curl -v http://localhost:8080/status/418
```

КОНФИДЕНЦИАЛЬНО

6. Что должно быть на скриншотах

№	Скриншот	Что должно быть видно
S1	Вкладка «Клиент» после GET http://example.com	URL, метод, статус 200, заголовки, тело
S2	Вкладка «Клиент» после POST на /echo	тело запроса и эхо в ответе
S3	Вкладка «Клиент» с ошибкой (неверный URL)	сообщение об ошибке в области ответа
S4	Вкладка «Сервер» запущена	зелёный индикатор, порт 8080
S5	curl -v http://localhost:8080/ в терминале	запрос/ответ, лог в GUI обновился
S6	curl -X POST -d "hello" .../echo	эхо «hello»
S7	curl .../status/418	статус 418
S8	curl .../nonexistent	404
S9	Лог сервера с 10 запросами	метод, путь, статус, время

S10	pytest tests/ -v	все тесты PASS
S11	flake8 src/ tests/	нет ошибок
S12	100 параллельных запросов (ab или скрипт)	все 200, GUI жив
S13	Диаграмма классов (PlantUML)	архитектура

7. Вопросы на защиту

7.1. Про HTTP

1. Чем HTTP/1.0 отличается от HTTP/1.1? Почему Host обязателен?
2. Что такое Content-Length? Что делать, если его нет?
3. Как работает Transfer-Encoding: chunked?
4. Чем GET и POST отличаются на уровне протокола?
5. Что такое keep-alive? Как сервер понимает, что соединение можно закрыть?
6. 301 vs 302 vs 307 — как клиент должен реагировать?
7. Почему HEAD — не «GET без тела»?
8. Что такое Content-Type и зачем charset?

КОНФИДЕНЦИАЛЬНО

7.2. Про сокеты

9. Что делает listen? Что такое backlog?
10. Чем accept отличается от connect?
11. Почему recv может вернуть меньше байт? Как правильно читать HTTP-ответ?
12. Что такое SIGPIPE в Python? Как его избежать?
13. Чем close отличается от shutdown?
14. Что такое SO_REUSEADDR? Зачем нужен?

7.3. Про Python и ООП

15. Почему Socket — контекстный менеджер? Что произойдёт без __exit__?
16. Почему Router хранит Callable, а не строку с именем функции?
17. Как вы обрабатываете исключения в __exit__?
18. Почему Request и Response — dataclass? Что это даёт?
19. Где в вашем коде isinstance? Можно ли его избежать?
20. Почему многопоточный сервер требует threading.Lock для логов?

7.4. Про GUI и многопоточность

21. Почему нельзя обновлять GUI из фонового потока в tkinter?
22. Как root.after решает проблему?
23. Что такое queue.Queue и зачем он между потоками?
24. Как остановить сервер, чтобы он не висел на accept?
25. Что будет, если пользователь закроет окно во время запроса?

7.5. Про безопасность

26. Что произойдёт, если клиент пришлёт заголовок 1 МБ? Как защищаетесь?
27. Что такое HTTP request smuggling?
28. Почему Content-Length и Transfer-Encoding одновременно — опасно?
29. Что такое path traversal (GET /../../etc/passwd)?
30. Почему нельзя использовать strcpy-аналоги при парсинге?

КОНФИДЕНЦИАЛЬНО

8. Критерии оценки

Критерий	Баллы
URL-парсер + unit-тесты (U1–U3)	10
HTTP-парсер + сериализатор (U4–U10)	15
Клиент GET/POST работает с реальным сервером (S1, S2)	15
Сервер отвечает на все маршруты (S4–S8)	15
Интеграционные тесты (I1–I5)	10
GUI: вкладки, фоновые потоки, обработка ошибок (S1–S4, S9)	15
Отчёт: диаграмма, скриншоты, анализ	10
Ответы на вопросы	10
Итого	**100**